← Índice de la Bible

estado: accepted · **fecha**: 2026-07-18 · **fecha-resolucion**: 2026-07-18

ADR-0005 — Lenguaje y runtime del núcleo de AURA

Contexto

El núcleo de AURA (orquestación, permisos, bus, módulos de cognición) necesita un lenguaje/runtime. La decisión condiciona ecosistema de IA disponible, latencia, sostenibilidad y contrataciones futuras (D19). Es una selección de tecnología con alternativas → 4 niveles (D3), decisión del CEO.

Alternativas consideradas

A. Económica — Python puro

Pros: máximo ecosistema de IA/ML; una sola lengua; adopción mínima. Contras: rutas calientes (audio, vectorial) pueden quedar cortas de rendimiento.

B. Balanceada — Python en el núcleo + módulos críticos optimizables (recomendada †)

Python para todo el núcleo y la lógica; se permite optimizar puntos calientes concretos (p. ej. vía extensiones nativas) solo si una medición lo exige (r13 ADR-0004). Pros: ecosistema de IA + camino de rendimiento sin cambiar de lengua; sostenible por equipo pequeño. Contras: dos toolchains en los puntos optimizados.

C. Profesional — Python + Rust en rutas calientes

Rust para audio/vectorial/concurrencia; Python para orquestación. Pros: rendimiento alto manteniendo el ecosistema IA. Contras: dos lenguas desde el inicio; más superficie de mantenimiento.

D. Enterprise — arquitectura políglota por servicios

Cada dominio en su mejor lengua, como servicios. Pros: máxima flexibilidad y escalado. Contras: complejidad operativa alta; sobredimensionado para un usuario (tensiona r13).

Decisión

Aprobada por el CEO (2026-07-18): Nivel Balanceado (B) — Python como lenguaje principal del proyecto, con optimización puntual medida. Regla del CEO: *"Solo se permitirá incorporar otros lenguajes cuando exista una necesidad técnica demostrable y documentada mediante una Alerta de Arquitectura."* Es decir, pasar a Rust/servicios (niveles C/D) exige Alerta de Arquitectura, no un simple ADR.

Consecuencias

Positivas (B): ecosistema IA completo; una lengua base; camino de rendimiento abierto. **Negativas:** optimización puntual añade toolchain local. **Riesgos aceptados:** si la latencia de 2.5 §5.1 no se alcanza en Python en rutas calientes, se escala a C por ADR.

Impacto (acta D11 — obligatorio)

Dimensión	Impacto
Seguridad	Neutro; Python maduro, dependencias auditables (SBOM, 3.4)
Privacidad	Positivo: voz y memoria locales en Python refuerzan ADR-0001
Modularidad	Positivo: contratos (3.3/3.4) independientes de la lengua
Costo	Bajo: sin licencias; equipo pequeño
Experiencia de usuario	Depende de latencia; B abre optimización sin rediseño
Mantenimiento futuro	Positivo: una lengua base; sustituible por servicios si escala

Trazabilidad

Capítulos a actualizar: 3.6 (registra la elección), 7.1 (estándares del lenguaje elegido), 7.3 (walking skeleton). Módulos de código afectados (futuro): todos — es la lengua base.